Check Your Setup

Not done yet? Start now. **No Linux machine?** Come and talk to us.

Setup guide: README.md at github.com/azurechen1203/microros-workshop

Docker & Workshop Files

1. `cd ~/microros_workshop`
2. `git pull`
3. `cd workshop && ./run.sh`
4. `source ~/microros_ws/install/local_setup.bash`
5. `ros2 pkg list | grep micro_ros`

Expect: `micro_ros_agent, _msgs, _setup`

Arduino IDE 2

Both menus should be listed:

1. Tools > Board > esp32
2. File > Examples > micro_ros_arduino

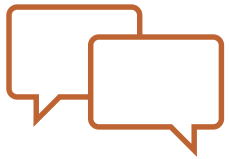
micro-ROS from Zero: Hands-On with ESP32

Bringing ROS 2 to Microcontrollers

Disclaimer



My first solo workshop



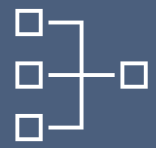
English is my second language



Not an expert, sharing what I wish I'd known

Why micro-ROS?

High-Level Layer



ROS 2

Nodes & topics over DDS

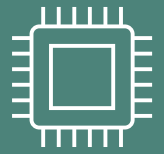
Perception

Localisation & SLAM

Planning

Mission Management

Low-Level Layer



Microcontroller

Arduino / STM32 firmware

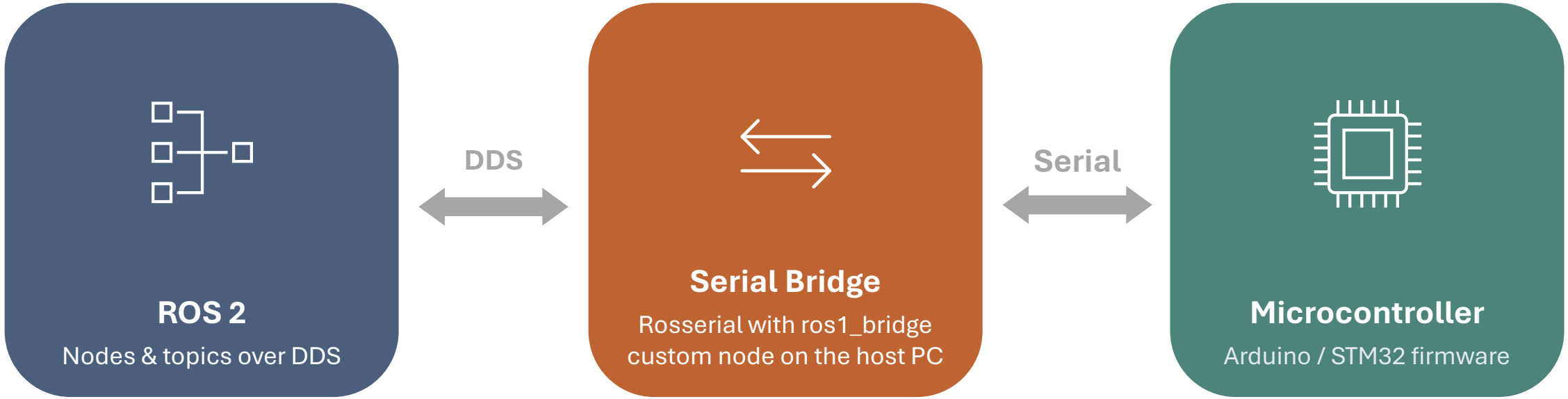
Motor Control (PWM)

Encoders

Sensors

GPIO / Peripherals

Before micro-ROS: Bridging ROS 2 and the MCU



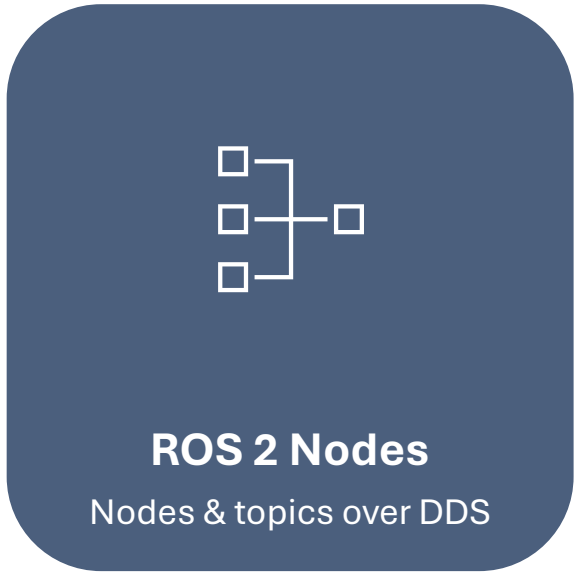
The catch: why this got painful

Change the serial format and the bridge parser must also change.

Every extra microcontroller needs its own bridge to maintain.

No native pub/sub, QoS or reuse of standard .msg. This adds latency and extra point or failure.

micro-ROS: Bridging ROS 2 and the MCU



ROS 2 Nodes

Nodes & topics over DDS

I speak Data Distribution Services.

My brain is only 520 KB



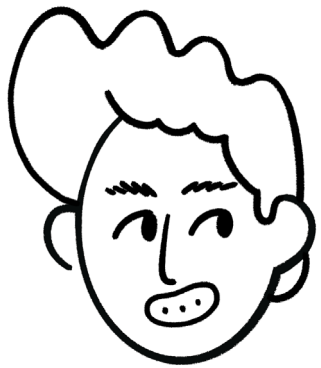
Microcontroller

Arduino / STM32 firmware

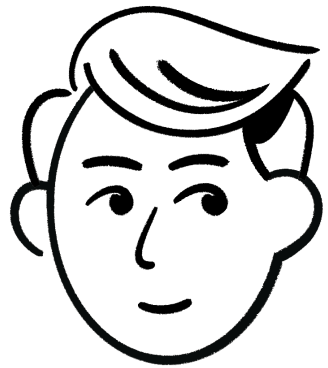
DDS

micro XRCE-DDS

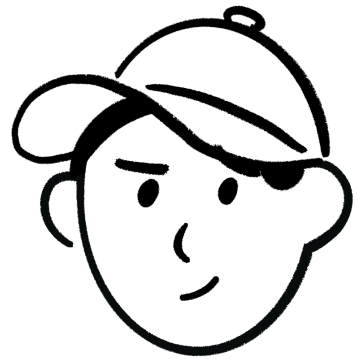
(eXtremely Resource Constrained Environments)



Fancy some scran?

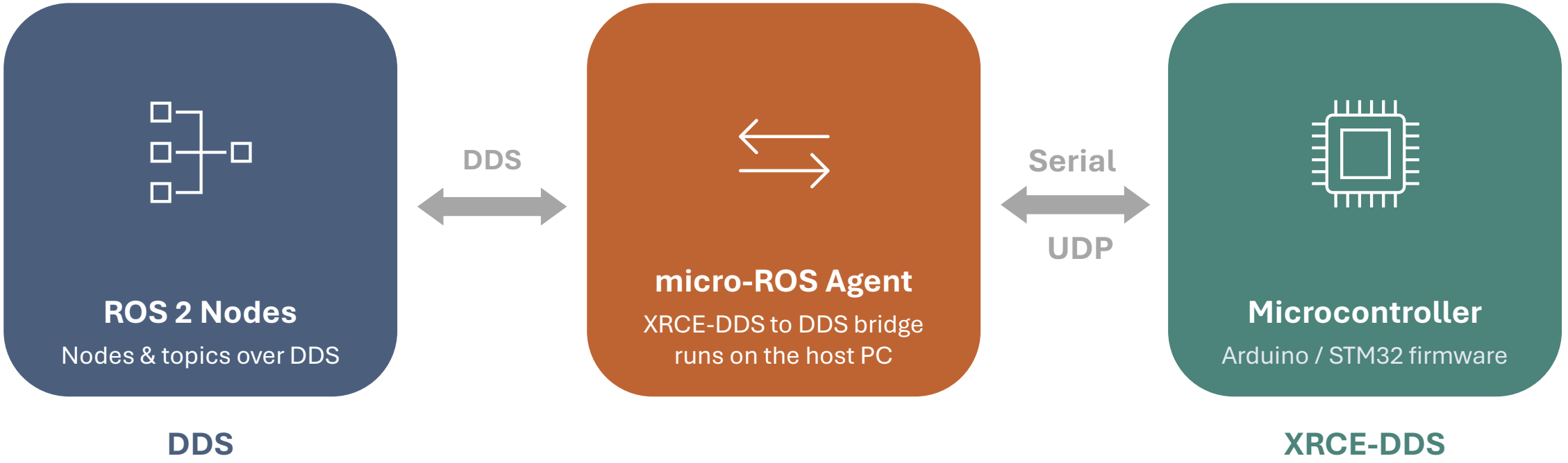


Scran means food

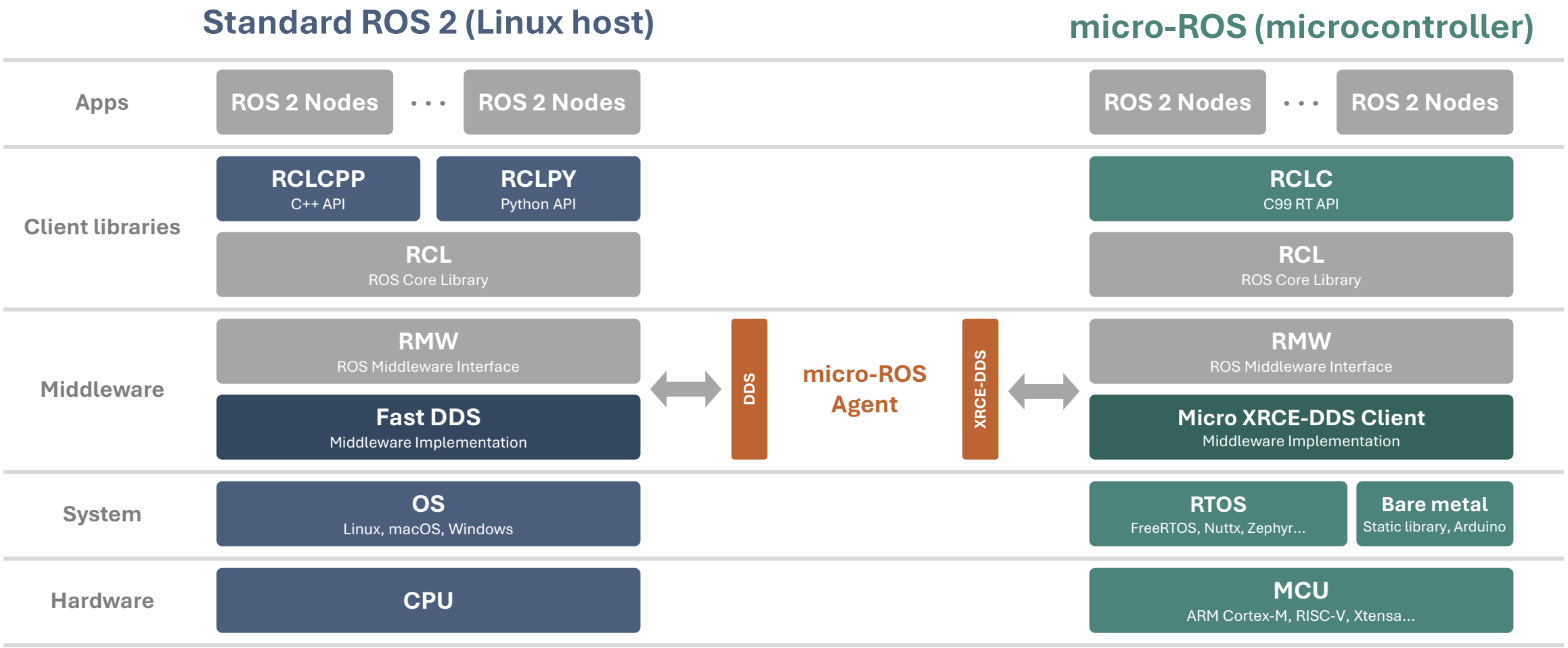


WHAT ?!?

micro-ROS Agent: Bridge to the ROS 2 World



Architecture: ROS2 vs micro-ROS



Supported Hardware & Build Options

Boards



ESP32



STM32



Pico RP2040

PJRC

Teensy 4.x



Arduino

Build paths



Arduino IDE
easiest start · today's path



PlatformIO
builds from source · more control



Native RTOS
ESP-IDF · Zephyr · production

micro-ROS Client Example

Open an example file: **File > Examples > micro_ros_arduino > micro-ros_subscriber**

micro-ros_subscriber.ino

```
#include <micro_ros_arduino.h>

#include <stdio.h>
#include <rcl/rcl.h>
#include <rcl/error_handling.h>
#include <rclc/rclc.h>
#include <rclc/executor.h>

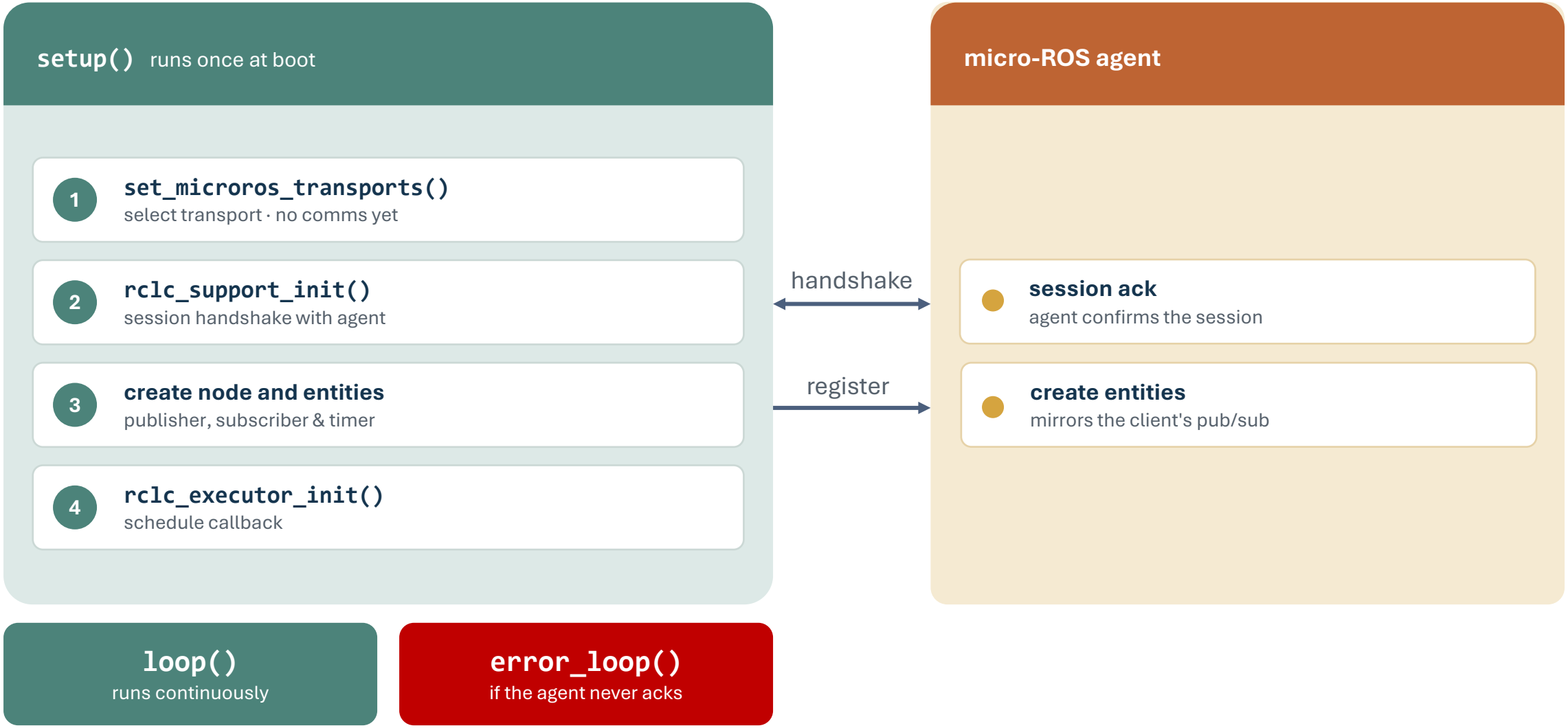
#include <std_msgs/msg/int32.h>

rcl_subscription_t subscriber;
std_msgs__msg__Int32 msg;
rclc_executor_t executor;
rclc_support_t support;
rcl_allocator_t allocator;
rcl_node_t node;
rcl_timer_t timer;

#define LED_PIN RGB_BUILTIN

#define RCCHECK(fn) {rcl_ret_t temp_rc = fn; if((temp_rc != RCL_RET_OK)){error_loop();}}
#define RCSOFTCHECK(fn) {rcl_ret_t temp_rc = fn; if((temp_rc != RCL_RET_OK)){}}
```

Anatomy of a micro-ROS Program

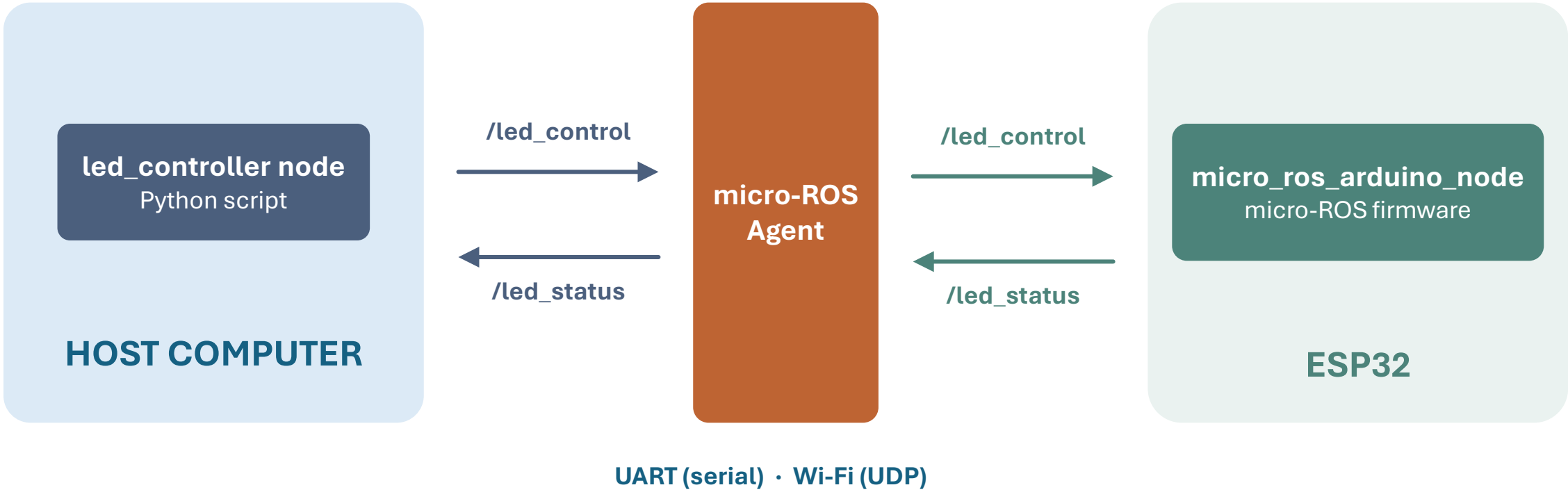


Hands-on Time!

First micro-ROS Client

Follow First micro-ROS Client in HANDOUT

Today’s Example: Nodes & Topics



ALREADY FAMILIAR

1

ROS 2 Package

The ROS 2 already known

ONE-TIME SETUP

2

micro-ROS Agent &

Bridge on the host
build once

ESP32 + Library

Arduino IDE setup
install once

WORKSHOP FOCUS

3

micro-ROS Client

Write & flash
every exercise

PART 1 - Foundations

1 Subscriber — LED control

2 Publisher — LED status

3 Work with strings

PART 2 - Debugging

4 Serial output

5 Serial output 2

6 ROS logs

PART 3 - Networking

7 ROS domain ID

8 Wi-Fi transport

9 Auto-reconnection

File System

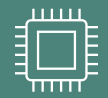
```
~/microros_workshop/
├── README.md
├── HANDOUT.md
└── workshop/
    ├── Dockerfile
    ├── run.sh
    ├── attach.sh
    │
    ├── firmware/
    │   ├── exercise1/
    │   │   └── exercise1.ino
    │   ├── ...
    │   └── exercise9/
    │       └── exercise9.ino
    │
    ├── ros2_ws/
    │   ├── src/
    │   │   └── led_demo/
    │   ├── build/
    │   ├── install/
    │   └── log/
    │
    └── microros_ws/
```



`ros2_ws/`
ROS 2
ROS 2 packages



`microros_ws/`
microROS Agent
bridges the board to ROS 2



`firmware/`
Microcontroller
Arduino .ino sketches

Reminders



Solutions are at the end of each section



Try to understand the code, not just copy it



Use any editor you like, not only nano



Save each exercise in firmware/

GOAL



EXERCISE 1 TASKS

1

Rename the topic

Rename the subscriber topic to `led_control`

2

Handle the command

In the callback: **`0` → LED off, `1` → LED on**

3

Flag errors in red

Set the error-loop **LED to red** to spot failures

Hands-on Time!

Exercise 1 · LED Control (Subscriber)

Follow Exercise 1 in HANDOUT

Lesson learnt

- An ESP32 subscriber driven by `ros2 topic pub`
- Device side: micro-ROS agent → serial → ESP32 → LED
- A solid red LED means the client is in `error_loop()`

Annoying parts

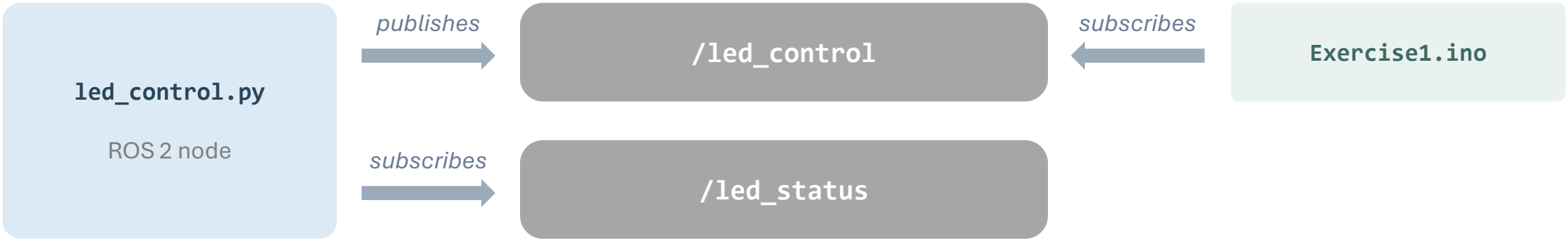
⚠ **Stop the agent before upload**
Both need the same serial port. Restart it after.

⚠ **Press reset after agent restarts**
`setup()` connects once at boot and never retries.

Looking Ahead

What if we don’t have access to the reset button, or can’t power cycle the board?

GOAL



ROS 2 TEST NODE TASKS

1

Create the package

Create `led_demo` package and add `led_control.py` with your namespace

2

Register and build

Add `led_control` to `setup.py`, then `colcon build`

3

Run the node

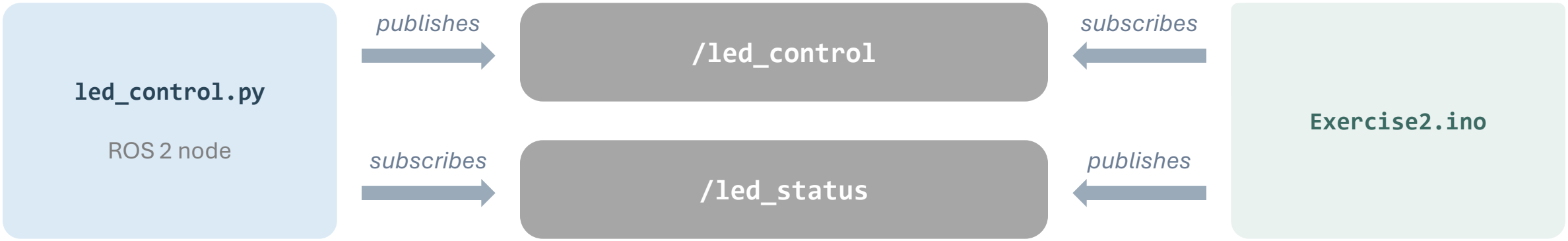
Run `led_control` and enter `0` or `1` to switch the LED

Hands-on Time!

Create ROS 2 Test Node

Follow Create ROS2 Test Node in HANDOUT

GOAL



EXERCISE 2 TASKS

1

Start from the example

Open `micro-ros_publisher` sketch

2

Wire up the publisher

Add publisher, timer & executor and bump the executor count

3

Publish the LED state

Timer callback sets

`led_status: 0 = off, 1 = on`

Hands-on Time!

Exercise 2 · LED Control (Publisher)

Follow Exercise 2 in HANDOUT

Strings on a Microcontroller

Integer

```
std_msgs__msg__Int32  status_msg
```

1

vs

String

```
std_msgs__msg__String status_msg
```

- `.data`
- `.capacity`
- `.size`

3 fields

Where the text is stored
points to a char buffer

How many bytes fit
the buffer size, fixed

How many letters are used now
changes with each message

⚠ **No dynamic memory on the MCU**
Give the string a fixed buffer: point `.data` at it, set `.capacity` to `sizeof(buf)`, and let `.size` track the current length.

Filling the String Buffer

What the code does

1. **Declare: create a buffer to reserve 4 bytes**

   `char status_msg_buf[4];`

   Then .capacity = 4 bytes

2. **setup(): .data points to the buffer**

   `status_msg.data.data = status_msg_buf;`

   Then .size = 0 letters

3. **LED on: the timer copies "on" in**

   `strcpy(status_msg.data.data, "on");`

   Then .size = strlen("on") = 2 letters (\0 not counted)

   LED off: the timer copies "off" in

   `strcpy(status_msg.data.data, "off");`

   Then .size = strlen("off") = 3 letters (all 4 bytes used)

4. **Publish: the timer sends the String**

   `rcl_publish(&publisher, &status_msg, NULL);`

   Sends the first .size letters: "off"



Hands-on Time!

Exercise 3 · Work with Strings

Follow Exercise 3 in HANDOUT

Debugging in micro-ROS

ARDUINO

`Serial.print()`

shows in the Serial Monitor

ROS 2

`get_logger().info()`

shows in the ROS 2 console

Looking Ahead

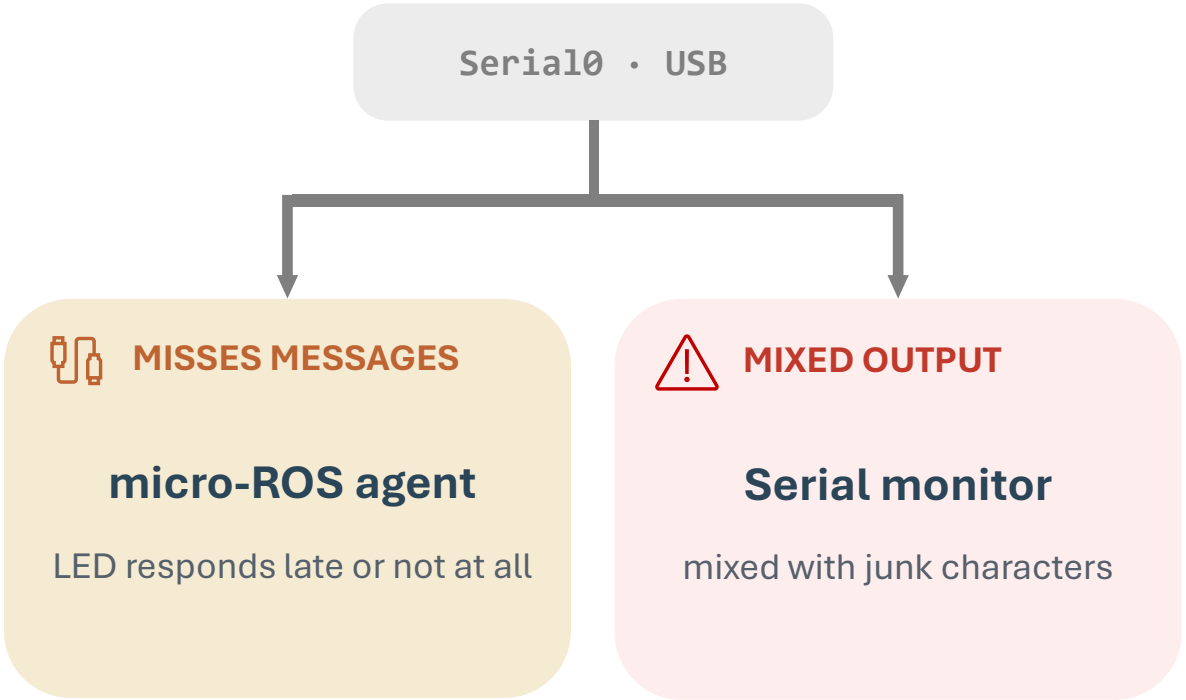
Do our usual debugging tools still work on micro-ROS?

Hands-on Time!

Exercise 4 · Serial Output

Follow Exercise 4 in HANDOUT

What Happened?



Workaround

`Serial0 · USB`

`Serial1 · TXRX`

CONNECTED

micro-ROS agent

Serial0 for ROS 2

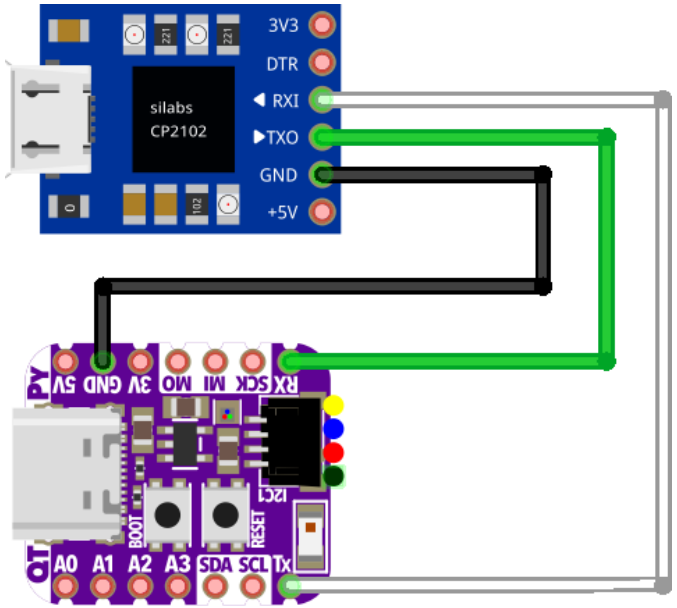
AVAILABLE

`Serial1.print()`

Serial1 for debug

```
Serial1.begin(115200, SERIAL_8N1, 7, 32);
Serial1.print("LED status ")
```

TTL-USB adapter



ROS Log

	String · LED status	Log · ROS log
Message type	`std_msgs/String`	`rcl_interfaces/Log`
Topic	`led_status`	`/rosout` shared by all nodes
Text buffer	`status_msg_buf[4]`	`log_msg_buf[64]`
Extra fields	none	`.level` severity, `.name` sender
Copy the text	`strcpy()`	`strncpy()` cuts off long text

Same steps as publishing a string: point at a buffer → copy the text in → set `.size` → publish

strcpy vs strncpy

`strcpy(dest, src)` Copies `src` and its ending `'\0'` into `dest`.

	What the code does	buf[4]	next variable	Result
1	**strcpy: the text fits** `strcpy(buf, "off");` 3 letters + \0 = 4 bytes, exactly the buffer	o f f \0	· ·	Fits
2	**strcpy: the text is too long** `strcpy(buf, "blink");` No limit: k and \0 overwrite the next variable	b l i n	k \0	Overwrites

`strncpy(dest, src, n)` Copies at most n characters, so it never writes past the limit.

	What the code does	buf[4]	next variable	Result
3	**strncpy: cut off, but no \0** `strncpy(buf, "blink", 4);` Stops at 4 bytes, so strlen() reads on past the end	b l i n	? ?	Reads past
4	**strncpy + \0: the handout's way** `strncpy(buf, "blink", 3); buf[3] = '\0';` Cut off to "bli", next variable untouched	b l i \0	· ·	Cut off, safe

Overflow gives no error on the ESP32. Rows 2 and 3 show up later as strange values, garbage text, or a crash and reset.

Hands-on Time!

Exercise 6 · ROS Logs

Follow Exercise 6 in HANDOUT

Debugging in micro-ROS: Summary

ARDUINO

`Serial.print()`

shows in the Serial Monitor

On micro-ROS

The agent already uses the serial port

Prints collide with micro-ROS traffic.

A clean fix needs a second UART.

ROS 2

`get_logger().info()`

shows in the ROS 2 console

On micro-ROS

Logs only appear once they reach ROS

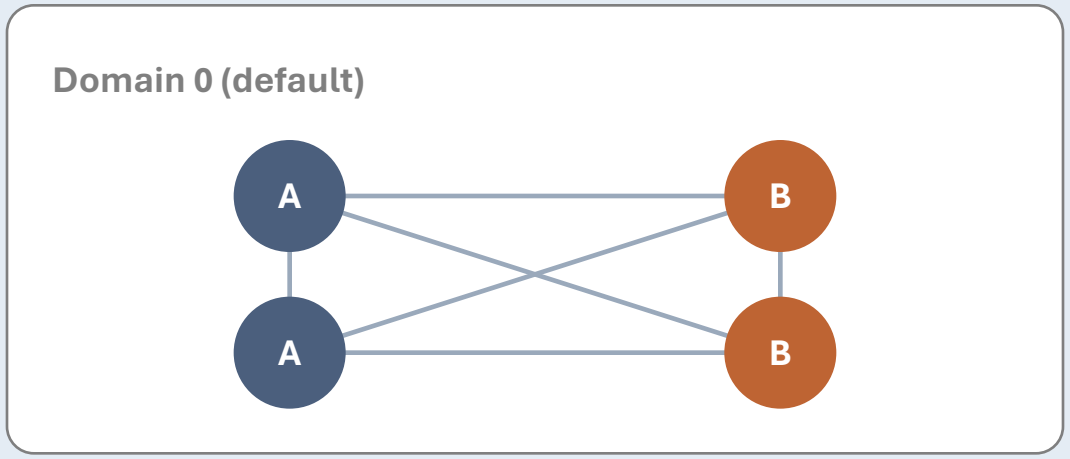
Agent down or link broken: no logs.

The bug stays invisible.

Why ROS 2 Needs Domain IDs

WITHOUT DOMAIN IDS

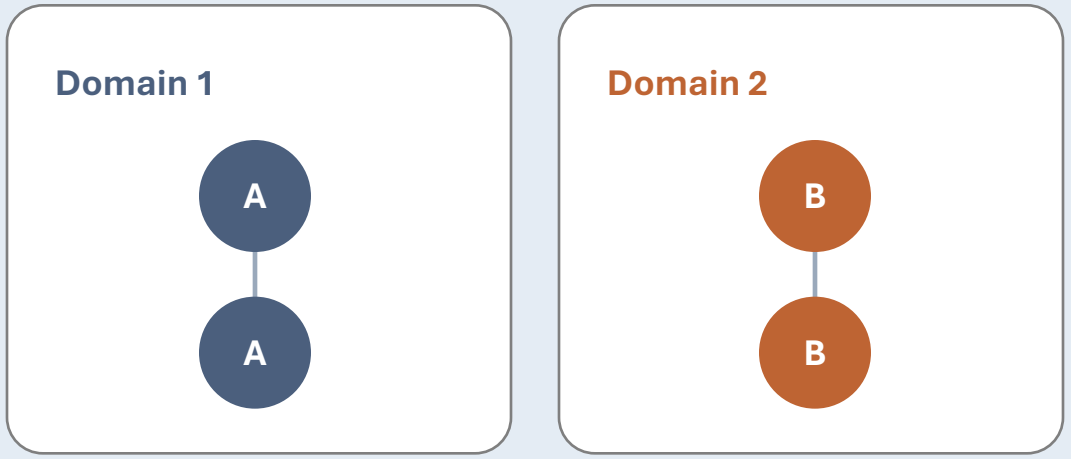
SHARED WI-FI NETWORK



Everyone sees everyone. Topics mix.

WITH DOMAIN IDS

SHARED WI-FI NETWORK

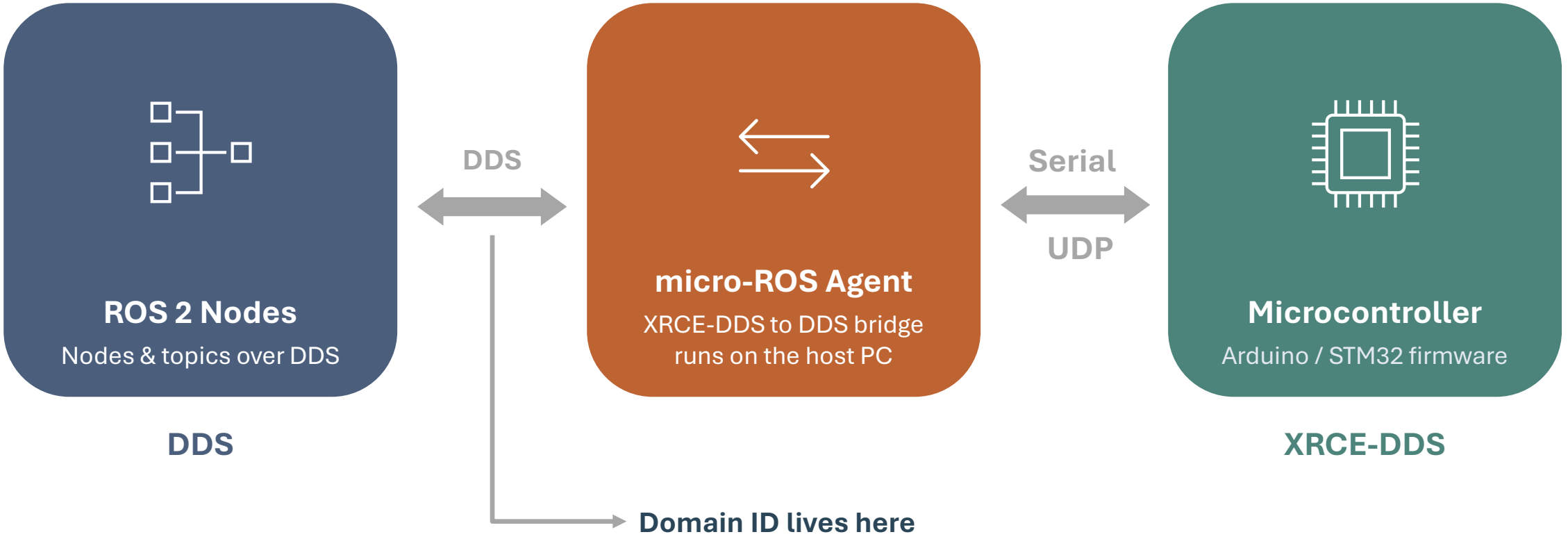


Nodes only see the **same** ID.

Set it with `ROS_DOMAIN_ID`

Default is `0`. Use any number from 0 to 101. Same ID: nodes can talk. Different ID: they're invisible to each other.

Where the Domain ID Lives



Hands-on Time!

Exercise 7 · ROS_DOMAIN_ID

Follow Exercise 7 in HANDOUT

Wi-Fi Transport

micro-ROS Client
`set_microros_wifi_transports()`



micro-ROS Agent
`ros2 run micro_ros_agent micro_ros_agent udp4 --port 8888`

EXERCISE 8 TASKS

1

Start a hotspot

Name it **`microros_<ID>`**, then get its IP with **`hostname -I`**

2

Add credentials and domain ID

Add a **`secrets.h`** tab, then set your domain ID as in Exercise 7

3

Switch to Wi-Fi

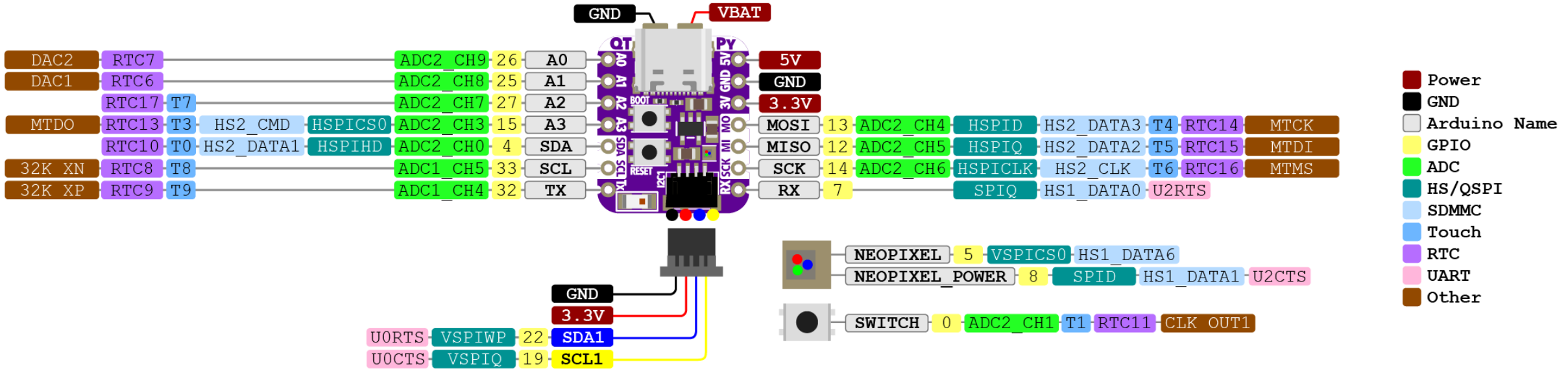
Use **`set_microros_wifi_transports()`** and run the agent with **`udp4`**

Hands-on Time!

Exercise 8 · Wi-Fi Transport

Follow Exercise 8 in HANDOUT

Wi-Fi and Analog Pins



Wi-Fi shares hardware with ADC2, so those pins can't read analog

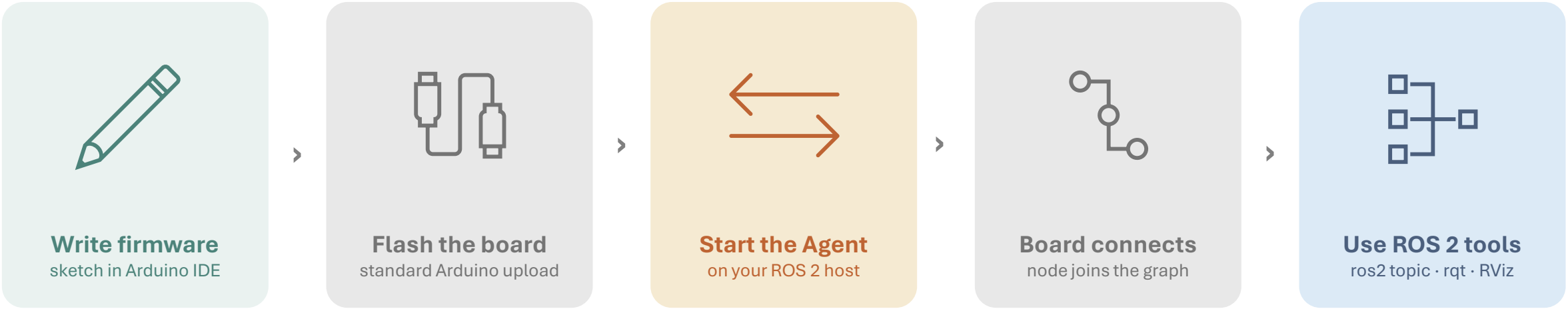
On a standard ESP32, use ADC1 pins for analog readings if they're free, or add an external ADC such as the ADS1115.

Hands-on Time!

Exercise 9 · Reconnection Handling

Follow Exercise 9 in HANDOUT

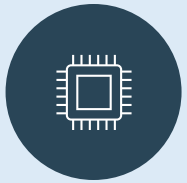
Workflow Recap



Serial vs. Wi-Fi: No Winner, Just Trade-offs

	Serial	**Wi-Fi**
Port occupation	Held by the micro-ROS agent, blocking both uploads and Serial.print()	Frees the serial port, Serial.print() works Limited pin for Analog reading
Debugging options	Serial1 + TTL-USB (extra hardware) /rosout	Serial.print() is available without extra hardware /rosout
Setup overhead	Minimal, plug in and go	Needs hotspot and WiFi credentials
Dependency	No network dependency	Depends on hotspot, may compete with internet access
Recovery after disconnection	Manual reset button / power cycle Auto reconnect if code is set up	Manual reset button / power cycle Auto reconnect if code is set up

When to use micro-ROS and when to not



Just Microcontroller



Standalone device

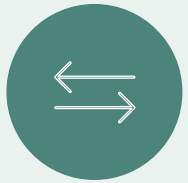


Simple sensor / actuator control



Quick prototype, minimal setup

VS



micro-ROS



MCU joins a ROS 2 network



Publish / subscribe with other nodes



Part of a larger robot system

Thank You

Feedback is welcome